

**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

Coordenação de tarefas - Parte 2

Sistemas Operacionais

Bernardo Souza Muniz.

08 de Maio de 2026

Engenharia de Telecomunicações - IFSC-SJ

Sumário

1. Questão 1	3
1.1. Implementação do Semáforo	3
1.2. Implementação do Mutex	5
2. Questão 2	7
2.1. Implementação utilizando Monitores	7
2.2. Implementação utilizando Mutex	9
2.3. Implementação utilizando Semáforo	11
3. Questão 3	13
4. Questão 4	15

1. Questão 1

Para a realização da questão 1, foi utilizado as implementações explícitas de Mutex e Semáforo através do pacote `java.util.concurrent`.

1.1. Implementação do Semáforo

Foi escolhido a implementação do exemplo do estacionamento em Java. Neste caso, foi utilizado a classe `Semaphore` e definido a quantidade de vagas disponíveis para entrada e saída de carros.

O resultado obtido está presente no código abaixo:

```
1 package ex1.semaforo;
2 import java.util.Random;
3 import java.util.concurrent.Semaphore;
4
5 public class Semaforo {
6     public static final int TOTAL_VAGAS = 5;
7     public static final int TOTAL_CARROS = 10;
8
9     private static final Semaphore semaforo = new Semaphore(TOTAL_VAGAS);
10    private static final Random r = new Random();
11
12    static void entrar_no_estacionamento(int arg){
13        System.out.println("Carro " + arg + " chegou e está procurando
14        vaga...");
15
16        try{
17            semaforo.acquire(); //Se não houver vaga, bloqueia
18
19            System.out.println("--> Carro " + arg + " ENTROU.");
20
21            Thread.sleep(5000 + (r.nextInt(3000)));
22
23        }catch (InterruptedException e){
24            Thread.currentThread().interrupt();
25        }finally{
26            System.out.println("<-- Carro " + arg + " SAIU. Liberando
27            vaga...");
28            semaforo.release();
29        }
30    }
31
32    public static void main(String[] args) {
33        Thread[] threads = new Thread[TOTAL_CARROS];
34
35        for (int i = 0; i < TOTAL_CARROS; i++) {
36            int id = i + 1;
37            threads[i] = new Thread(() -> entrar_no_estacionamento(id));
38            try {
39                Thread.sleep(600);
40            } catch (InterruptedException e) {
```

```

40         e.printStackTrace();
41     }
42     threads[i].start();
43
44 }
45
46 for (int i = 0; i < TOTAL_CARROS; i++) {
47     try {
48         threads[i].join();
49     } catch (InterruptedException e) {
50         Thread.currentThread().interrupt();
51     }
52 }
53
54 System.out.println("\nEstacionamento encerrado.\n");
55 }
56
57 }

```

Ao realizar a execução do código, foi possível observar que a premissa do semáforo foi bem executada, uma vez que a ordem de chegada de cada carro e o tempo de espera é feita de maneira síncrona, sem o ocasionamento de um carro estacionar em uma vaga que está ocupada:

Figura 1: Elaborada pelo Autor

```

● bernardo@debian:~/Downloads/java$ cd /home/bernardo/Downloads/java ; /usr/bin/env /usr/lib/jvm/
o/.config/Code/User/workspaceStorage/ec79a513c0e4709f0e75260fc3c9d6c7/redhat.java/jdt_ws/java_7ae
Carro 1 chegou e está procurando vaga...
--> Carro 1 ENTROU.
Carro 2 chegou e está procurando vaga...
--> Carro 2 ENTROU.
Carro 3 chegou e está procurando vaga...
--> Carro 3 ENTROU.
Carro 4 chegou e está procurando vaga...
--> Carro 4 ENTROU.
Carro 5 chegou e está procurando vaga...
--> Carro 5 ENTROU.
Carro 6 chegou e está procurando vaga...
Carro 7 chegou e está procurando vaga...
Carro 8 chegou e está procurando vaga...
Carro 9 chegou e está procurando vaga...
Carro 10 chegou e está procurando vaga...
<-- Carro 1 SAIU. Liberando vaga...
--> Carro 6 ENTROU.
<-- Carro 2 SAIU. Liberando vaga...
--> Carro 7 ENTROU.
<-- Carro 4 SAIU. Liberando vaga...
--> Carro 8 ENTROU.
<-- Carro 3 SAIU. Liberando vaga...
--> Carro 9 ENTROU.
<-- Carro 5 SAIU. Liberando vaga...
--> Carro 10 ENTROU.
<-- Carro 6 SAIU. Liberando vaga...
<-- Carro 7 SAIU. Liberando vaga...
<-- Carro 9 SAIU. Liberando vaga...
<-- Carro 8 SAIU. Liberando vaga...
<-- Carro 10 SAIU. Liberando vaga...

Estacionamento encerrado.

```

Execução do algoritmo de Semáforo implementando a versão em Java

1.2. Implementação do Mutex

Para implementação do Mutex em Java, foi escolhido o exemplo do contador.

```
1 package ex1.mutex;
2 import java.util.concurrent.locks.ReentrantLock;
3
4 public class Mutex{
5     public static final int NUM_THREADS = 100;
6     public static final int NUM_STEPS = 100000;
7     private static int sum = 0;
8     private static final ReentrantLock lock = new ReentrantLock();
9
10    static void threadBody(int id){
11        long my_id = (long)id;
12
13        for ( int i = 0; i < NUM_STEPS; i++){
14            lock.lock();
15
16            try {
17                sum += 1;
18                if (sum % 100000 == 0){
19                    System.out.println("Thread " + my_id + " incrementou.
20Sum: " + sum);
21                }
22            } finally {
23                lock.unlock();
24            }
25        }
26    }
27
28    public static void main(String[] args) {
29        Thread[] threads = new Thread[NUM_THREADS];
30        long expected = (long)NUM_THREADS * NUM_STEPS;
31
32        for (int i = 0; i < NUM_THREADS; i++) {
33            int id = i + 1;
34            threads[i] = new Thread(() -> threadBody(id));
35            try {
36                threads[i].sleep(100);
37            } catch (InterruptedException e) {
38                Thread.currentThread().interrupt();
39            }
40            threads[i].start();
41        }
42
43        for (int i = 0; i < NUM_THREADS; i++) {
44            try {
45                threads[i].join();
46            } catch (InterruptedException e) {
47                Thread.currentThread().interrupt();
48            }
49        }
50
51        System.out.println("\n--- Resultado: Pthread Mutex ---");
52        System.out.println("Valor esperado: " + expected);
```

```

53         System.out.println("Valor obtido: " + sum);
54         System.out.println("Diferença: " + (expected-sum));
55     }
56 }
57 }

```

Ao realizar a execução do código, foi possível verificar que a implementação do Mutex permitiu com que não houvesse condição de corrida entre as threads de incremento, garantindo o valor correto da variável sum.

Figura 2: Elaborada pelo Autor

```

Thread 74 incrementou. Sum: 7400000
Thread 75 incrementou. Sum: 7500000
Thread 76 incrementou. Sum: 7600000
Thread 77 incrementou. Sum: 7700000
Thread 78 incrementou. Sum: 7800000
Thread 79 incrementou. Sum: 7900000
Thread 80 incrementou. Sum: 8000000
Thread 81 incrementou. Sum: 8100000
Thread 82 incrementou. Sum: 8200000
Thread 83 incrementou. Sum: 8300000
Thread 84 incrementou. Sum: 8400000
Thread 85 incrementou. Sum: 8500000
Thread 86 incrementou. Sum: 8600000
Thread 87 incrementou. Sum: 8700000
Thread 88 incrementou. Sum: 8800000
Thread 89 incrementou. Sum: 8900000
Thread 90 incrementou. Sum: 9000000
Thread 91 incrementou. Sum: 9100000
Thread 92 incrementou. Sum: 9200000
Thread 93 incrementou. Sum: 9300000
Thread 94 incrementou. Sum: 9400000
Thread 95 incrementou. Sum: 9500000
Thread 96 incrementou. Sum: 9600000
Thread 97 incrementou. Sum: 9700000
Thread 98 incrementou. Sum: 9800000
Thread 99 incrementou. Sum: 9900000
Thread 100 incrementou. Sum: 10000000

--- Resultado: Pthread Mutex ---
Valor esperado: 10000000
Valor obtido: 10000000
Diferença: 0
aluno: java$

```

Execução do algoritmo do Contador implementando a versão em Java

2. Questão 2

A questão 2 consistia em solucionar o problema de condição de corrida (*Race Condition*) no sistema de pagamento bancário. Para isso, foi implementado em Java os algoritmos de Mutex, Monitoramento e Semáforo. O resultado final após a execução de cada código, é obter o valor final de R\$ 180 de saldo bancário.

2.1. Implementação utilizando Monitores

Ao realizar a implementação do sistema de pagamento bancário utilizando Monitores, foi obtido o seguinte resultado:

```
1 package exemplo2.monitor;
2
3 public class MonitorConta {
4     public static int saldo = 100;
5
6     static synchronized void depositar(int arg){
7         int valor = arg;
8
9         try {
10             Thread.sleep(0);
11             int temp = saldo;
12             temp += valor;           // Passo 2: Soma
13             saldo = temp;           // Passo 3: Escrita
14
15         } catch (InterruptedException e){
16             Thread.currentThread().interrupt();
17         }
18
19     }
20
21     public static void main(String[] args) {
22         int v1 = 50, v2 = 30;
23         Thread t1 = new Thread(()-> depositar(v1));
24         Thread t2 = new Thread(()-> depositar(v2));
25
26         t1.start();
27         t2.start();
28
29         try {
30             t1.join();
31             t2.join();
32         } catch (InterruptedException e) {
33             Thread.currentThread().interrupt();
34         }
35
36         System.out.println("Saldo final esperado: 180");
37         System.out.println("Saldo final obtido: " + saldo);
38
39     }
40 }
41 }
```

Ao executar o código mais de uma vez, foi obtido o valor correto de saldo bancário:

Figura 3: Elaborada pelo Autor



```
PROBLEMS (3) OUTPUT DEBUG CONSOLE TERMINAL PORTS

• aluno: SOP$ cd /home/aluno/SOP ; /usr/bin/env /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java -X
/redhat.java/jdt_ws/SOP_7a99e70f/bin exemplo2.monitor.MonitorConta
Saldo final esperado: 180
Saldo final obtido: 180
• aluno: SOP$ cd /home/aluno/SOP ; /usr/bin/env /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java -X
/redhat.java/jdt_ws/SOP_7a99e70f/bin exemplo2.monitor.MonitorConta
Saldo final esperado: 180
Saldo final obtido: 180
• aluno: SOP$ cd /home/aluno/SOP ; /usr/bin/env /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java -X
/redhat.java/jdt_ws/SOP_7a99e70f/bin exemplo2.monitor.MonitorConta
Saldo final esperado: 180
Saldo final obtido: 180
○ aluno: SOP$
```

Execução do algoritmo de conta bancária utilizando Monitores em Java

2.2. Implementação utilizando Mutex

Ao realizar a implementação do sistema de pagamento bancário utilizando Mutex, foi obtido o seguinte resultado:

```
1 package exemplo2.mutex;
2 import java.util.concurrent.locks.ReentrantLock;
3
4 public class MutexConta {
5     private static final ReentrantLock lock = new ReentrantLock();
6     public static int saldo = 100;
7
8     static void depositar(int arg){
9         int valor = arg;
10        lock.lock();
11
12        try {
13            Thread.sleep(100);
14            int temp = saldo;
15            temp += valor;           // Passo 2: Soma
16            saldo = temp;           // Passo 3: Escrita
17
18        }catch (InterruptedException e){
19            Thread.currentThread().interrupt();
20        }finally {
21            lock.unlock();
22        }
23    }
24 }
25
26 public static void main(String[] args) {
27     int v1 = 50, v2 = 30;
28     Thread t1 = new Thread(()-> depositar(v1));
29     Thread t2 = new Thread(()-> depositar(v2));
30
31     t1.start();
32     t2.start();
33
34     try {
35         t1.join();
36         t2.join();
37     }catch (InterruptedException e) {
38         Thread.currentThread().interrupt();
39     }
40
41     System.out.println("Saldo final esperado: 180");
42     System.out.println("Saldo final obtido: " + saldo);
43 }
44 }
```

Ao executar o código mais de uma vez, foi obtido o valor correto de saldo bancário:

Figura 4: Elaborada pelo Autor

```
• aluno: SOP$ cd /home/aluno/SOP ; /usr/bin/env /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java -XX:+ShowCodeD
rage/6f1217aed74bb8bae9419f13c13d8d1c/redhat.java/jdt_ws/SOP_7a99e70f/bin exemplo2.mutex.MutexConta
Saldo final esperado: 180
Saldo final obtido: 180
• aluno: SOP$ cd /home/aluno/SOP ; /usr/bin/env /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java -XX:+ShowCodeD
rage/6f1217aed74bb8bae9419f13c13d8d1c/redhat.java/jdt_ws/SOP_7a99e70f/bin exemplo2.mutex.MutexConta
Saldo final esperado: 180
Saldo final obtido: 180
• aluno: SOP$ cd /home/aluno/SOP ; /usr/bin/env /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java -XX:+ShowCodeD
rage/6f1217aed74bb8bae9419f13c13d8d1c/redhat.java/jdt_ws/SOP_7a99e70f/bin exemplo2.mutex.MutexConta
Saldo final esperado: 180
Saldo final obtido: 180
• aluno: SOP$ /usr/bin/env /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java -XX:+ShowCodeDetailsInExceptionMess
e9419f13c13d8d1c/redhat.java/jdt_ws/SOP_7a99e70f/bin exemplo2.mutex.MutexConta
• ^Caluno: SOP$ cd /home/aluno/SOP ; /usr/bin/env /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java -XX:+ShowCod
torage/6f1217aed74bb8bae9419f13c13d8d1c/redhat.java/jdt_ws/SOP_7a99e70f/bin exemplo2.mutex.MutexConta
Saldo final esperado: 180
Saldo final obtido: 180
```

Execução do algoritmo de conta bancária utilizando Mutex em Java

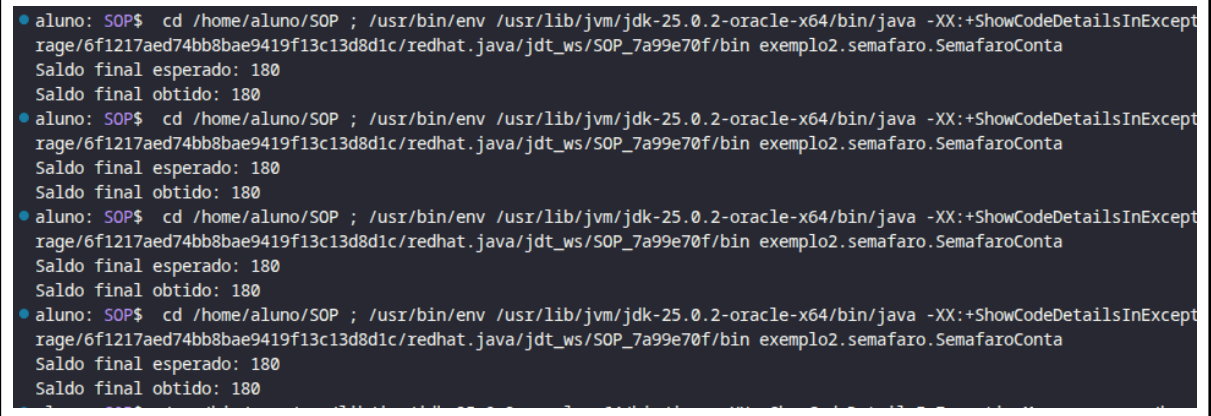
2.3. Implementação utilizando Semáforo

Ao realizar a implementação do sistema de pagamento bancário utilizando Semáforos, foi obtido o seguinte resultado:

```
1 package exemplo2.semáforo;
2 import java.util.concurrent.Semaphore;
3
4 public class SemáforoConta {
5     private static final Semaphore semaforo = new Semaphore(2);
6     public static int saldo = 100;
7
8     static void depositar(int arg){
9         int valor = arg;
10
11
12         try {
13             semaforo.acquire();
14             Thread.sleep(100);
15             int temp = saldo;
16             temp += valor;           // Passo 2: Soma
17             saldo = temp;           // Passo 3: Escrita
18
19         } catch (InterruptedException e){
20             Thread.currentThread().interrupt();
21         } finally {
22             semaforo.release();
23         }
24     }
25 }
26
27 public static void main(String[] args) {
28     int v1 = 50, v2 = 30;
29     Thread t1 = new Thread(()-> depositar(v1));
30     Thread t2 = new Thread(()-> depositar(v2));
31
32     t1.start();
33     t2.start();
34
35     try {
36         t1.join();
37         t2.join();
38     } catch (InterruptedException e) {
39         Thread.currentThread().interrupt();
40     }
41
42     System.out.println("Saldo final esperado: 180");
43     System.out.println("Saldo final obtido: " + saldo);
44
45 }
46 }
47 }
```

Ao executar o código mais de uma vez, foi obtido o valor correto de saldo bancário:

Figura 5: Elaborada pelo Autor



```
aluno: SOP$ cd /home/aluno/SOP ; /usr/bin/env /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java -XX:+ShowCodeDetailsInException /6f1217aed74bb8bae9419f13c13d8d1c/redhat.java/jdt_ws/SOP_7a99e70f/bin exemplo2.semapharo.SemafaroConta
Saldo final esperado: 180
Saldo final obtido: 180
aluno: SOP$ cd /home/aluno/SOP ; /usr/bin/env /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java -XX:+ShowCodeDetailsInException /6f1217aed74bb8bae9419f13c13d8d1c/redhat.java/jdt_ws/SOP_7a99e70f/bin exemplo2.semapharo.SemafaroConta
Saldo final esperado: 180
Saldo final obtido: 180
aluno: SOP$ cd /home/aluno/SOP ; /usr/bin/env /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java -XX:+ShowCodeDetailsInException /6f1217aed74bb8bae9419f13c13d8d1c/redhat.java/jdt_ws/SOP_7a99e70f/bin exemplo2.semapharo.SemafaroConta
Saldo final esperado: 180
Saldo final obtido: 180
aluno: SOP$ cd /home/aluno/SOP ; /usr/bin/env /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java -XX:+ShowCodeDetailsInException /6f1217aed74bb8bae9419f13c13d8d1c/redhat.java/jdt_ws/SOP_7a99e70f/bin exemplo2.semapharo.SemafaroConta
Saldo final esperado: 180
Saldo final obtido: 180
```

Execução do algoritmo de conta bancária utilizando Semáforos em Java

3. Questão 3

A questão 3 consistia em alterar o exemplo da pizzeria implementado em Monitores. Foi sugerido que o cozinheiro conseguisse colocar na mesa até 2 pizzas para o entregador levar. Para isso, foi utilizado a estrutura de dados Fila (*Queue*) e seus demais métodos.

- **Classe Mesa:**

```
1 package exemplo3.Monitor;
2
3 import java.util.LinkedList;
4 import java.util.Queue;
5
6
7 class Mesa {
8     private String pizza;
9     //private boolean mesaVazia = true;
10    public static Queue<String> pizzas = new LinkedList<>(); //a fila de
    pizzas começa vazia
11
12
13    // Monitor: Garante que só um mexe na mesa por vez
14    public synchronized void colocarPizza(String nomePizza) throws
    InterruptedException {
15        //se a fila de pizzas consegue armazenar 2 pizzas, o cozinheiro
        continua colocando pizzas, se não, ele espera
16        while (pizzas.size() == 2) {
17            System.out.println("-> Mesa cheia! Cozinheiro esperando...");
18            wait();
19        }
20
21        pizzas.add(nomePizza);
22        System.out.println("Cozinheiro colocou a pizza de: " + pizzas);
23        // Avisa o entregador que tem pizza pronta
24        notifyAll();
25    }
26
27    public synchronized String retirarPizza() throws InterruptedException
    {
28        while (pizzas.isEmpty()) {
29            System.out.println("-> Mesa vazia! Entregador esperando...");
30            wait();
31        }
32
33        String p = pizzas.poll();
34        System.out.println("Entregador levou a pizza de: " + p);
35        // Avisa o cozinheiro que a mesa liberou
36        notifyAll();
37        return p;
38    }
39 }
```

- **Classe Pizzaria:**

```

1 package exemplo3.Monitor;
2
3 public class Pizzaria {
4     public static void main(String[] args) {
5         Mesa mesa = new Mesa();
6
7
8         // Thread do Cozinheiro
9         new Thread(() -> {
10
11             try {
12                 String[] cardapio = {"Calabresa", "Mussarela",
13 "Portuguesa", "Marguerita", "Frango"};
14                 for (String p : cardapio) {
15                     mesa.colocarPizza(p);
16                     Thread.sleep(0); // Cozinheiro rápido
17                 }
18             } catch (InterruptedException e) {}
19         }).start();
20
21         // Thread do Entregador
22         new Thread(() -> {
23             try {
24                 for (int i = 0; i < 3; i++) {
25                     mesa.retirarPizza();
26                     Thread.sleep(3000); //Entregador lento
27                 }
28             } catch (InterruptedException e) {}
29         }).start();
30     }
31 }

```

Ao executar o código, para o caso aonde o cozinheiro é mais rápido que o entregador, foi possível observar que o *buffer* de até duas pizzas para o cozinheiro estava funcionando corretamente:

Figura 6: Elaborada pelo Autor

```

aluno: SOP$ cd /home/aluno/SOP ; /usr/bin/env /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java -XX:+ShowCodeE
rage/6f1217aed74bb8bae9419f13c13d8d1c/redhat.java/jdt_ws/SOP_7a99e70f/bin exemplo3.Monitor.Pizzaria
Cozinheiro colocou a pizza de: [Calabresa]
Entregador levou a pizza de: Calabresa
Cozinheiro colocou a pizza de: [Mussarela]
Cozinheiro colocou a pizza de: [Mussarela, Portuguesa]
-> Mesa cheia! Cozinheiro esperando...
Entregador levou a pizza de: Mussarela
Cozinheiro colocou a pizza de: [Portuguesa, Marguerita]
-> Mesa cheia! Cozinheiro esperando...
Entregador levou a pizza de: Portuguesa
Cozinheiro colocou a pizza de: [Marguerita, Frango]
aluno: SOP$

```

Execução do algoritmo da pizzaria com *buffer* maior para o cozinheiro

4. Questão 4

As principais desvantagens e limitações na utilização de Semáforos podem ser destacadas como:

- **Possibilidade de bloqueio mútuo:** Na implementação de uso de Semáforos, pode ocorrer o caso em que um processo contendo Thread A que fique esperando por uma Thread B e outro processo é contendo a Thread B esperando por A. Neste caso é gerado um impasse, pois temos uma thread esperando pela outra para iniciar sua execução.
- **Necessidade de liberar e fechar o semáforo manualmente:** Para cenários maiores e complexos, o programador do código pode acabar esquecendo de liberar ou bloquear determinada Thread, causando resultados inconsistentes e inesperados.
- **Utilização de variável global:** Nos exemplos de semáforos sempre era necessário o uso de uma variável global sendo acessível pelas tarefas em questão. Como se tratava de apenas exemplos simples, não houve problemas de alteração do valor da variável. Em cenários maiores e mais complexos, manter uma variável global sendo acessível por várias Threads simultaneamente e por demais funções presentes no código, pode causar valores inesperados.